

LLM FOUNDATIONS

Token · Tokenization · Context · Context Window · Embedding · Cosine Similarity · Top-N
Temperature · Top-K · Sentiment Value · Hallucination · Reasoning · Zero-shot · Few-shot

1. Token

Definition: A token is the smallest unit of text an LLM reads and writes. It can be a whole word, part of a word, a space-prefixed word, a punctuation mark, or a digit group.

Why not letters or words?

A neural network only does arithmetic on numbers, so text must first become a sequence of integers. We have three choices for the unit of text, and each has a clear trade-off:

- **Characters:** tiny vocabulary (~100 symbols) but very long sequences. "unbelievable" becomes 12 steps, and the model must learn spelling before meaning. Slow and expensive.
- **Whole words:** short sequences but a huge vocabulary (millions of words, names, typos, SKUs). Any unseen word ("Paytm", "refund") becomes an unknown symbol.
- **Sub-words (tokens):** the engineering middle ground. Frequent words stay whole; rare words are built from reusable pieces. Vocabularies are typically 32K to 200K tokens.

So a token is a **compression unit**: the model's "atom" of meaning, and also the **unit of billing, latency, and memory**. Every LLM API charges per token and every limit (context window, max output) is measured in tokens.

Mathematics

English rule of thumb: 1 token $\approx$ 4 characters $\approx$ 0.75 words

$$\text{Cost} = (T_{\text{in}} \times \text{Price}_{\text{in}} + T_{\text{out}} \times \text{Price}_{\text{out}}) / 1,000,000$$

$$\text{Latency} \approx \text{Time-to-first-token} + T_{\text{out}} \times (\text{time per output token})$$

Prices are quoted per million tokens; output tokens usually cost 3–5× input tokens

Worked example (illustrative prices: \$3 / M input, \$15 / M output): a support bot handles 50,000 tickets per day. Each call sends 800 input tokens and returns 150 output tokens.

- Input: $50,000 \times 800 = 40,000,000$ tokens $\rightarrow 40 \times \$3 =$ **\$120 / day**
- Output: $50,000 \times 150 = 7,500,000$ tokens $\rightarrow 7.5 \times \$15 =$ **\$112.50 / day**
- Total $\approx$ **\$232.50 / day $\approx$ \$7,000 / month**. Cutting the system prompt by 300 tokens saves 15M tokens/day, i.e. \$45/day, just by editing text.

Real example

The same sentence costs different amounts in different languages. "Where is my order?" is about 5 tokens in English, but the Hindi or Kannada equivalent can take 2–4× more tokens on many tokenizers, because those scripts appeared less in tokenizer training data. For an Indian multilingual support bot this directly multiplies cost and reduces how much text fits in the context window.

Industry use cases

Industry	Problem statement	How this concept solves it
Fintech (India)	Multilingual bot bill was 3× the forecast after launching Hindi and Tamil support.	Team measured tokens per language, budgeted per-language cost, and chose a model whose tokenizer is efficient on Indic scripts.
SaaS	Unpredictable LLM spend per customer made pricing plans unprofitable.	Metered usage by tokens (input + output) per tenant and set per-plan token quotas.
E-commerce	Product-description generation was slow at peak sale time.	Capped max_tokens for output; since latency scales with output tokens, p95 latency dropped sharply.

2. Tokenization

Definition: Tokenization is the deterministic process that converts raw text into a sequence of token IDs (and back). It happens before the model sees anything.

Input text: "Refund not received, unbelievable!"

Ref	und	·not	·received	,	·un	believ	able	!
4032	1263	539	4036	11	653	31141	481	13

Token IDs (illustrative) → the model never sees letters, only these integers

4 words → 9 tokens · '·' marks a leading space (part of the token) · rare word 'unbelievable' split into 3 sub-words

How is the vocabulary built? (Byte-Pair Encoding)

Most modern LLMs use **BPE** or a close variant. The idea is pure compression: start from bytes/characters, then repeatedly merge the most frequent adjacent pair into a new symbol, until the vocabulary reaches a target size.

$(a^*, b^*) = \operatorname{argmax}_{(a, b)} \operatorname{count}(a, b) \rightarrow \text{add new token "ab"}$
Repeat until $|\text{Vocabulary}| = V$ (e.g. $V = 100,000$)
Frequent strings become single tokens; rare strings stay split

Worked example: corpus = "low", "lower", "lowest". Start with characters l, o, w, e, r, s, t. The pair (l, o) appears 3 times → merge into "lo". Then (lo, w) appears 3 times → merge into "low". Now "lower" = [low, e, r] and "lowest" = [low, e, s, t]. The model learns "low" once and reuses it.

Trade-off: larger $V \rightarrow$ fewer tokens per sentence (cheaper, longer effective context) but a bigger embedding table.
Tokens per text $\approx$ characters $\div$ (average characters per token).

Why engineers must care

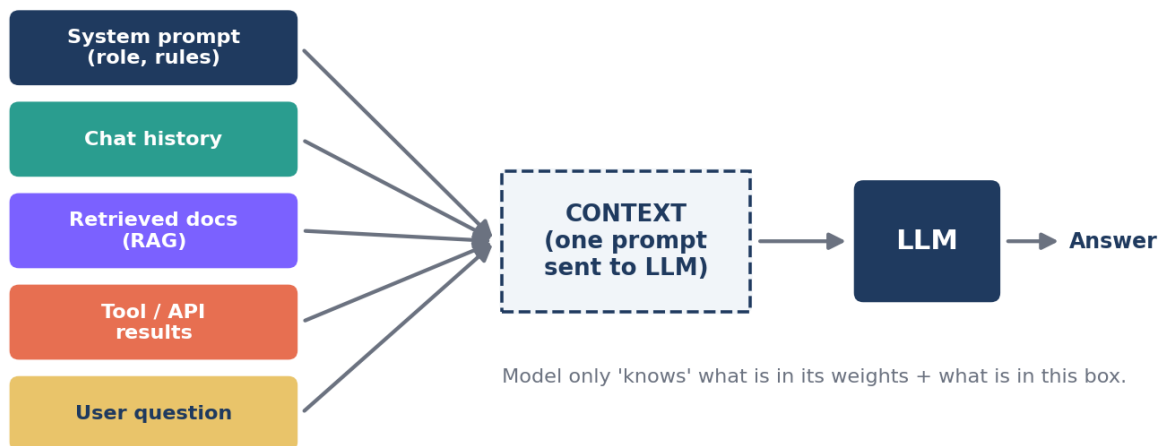
- **Tokenizers are model-specific.** The same text gives different token counts on different model families. Always count with the target model's tokenizer (e.g. [tiktoken](#) for OpenAI models, the provider's token-count API for others).
- **Numbers and code split oddly.** "2024-09-18" may be 5+ tokens; long numbers are split into chunks, which is one reason LLMs are weak at exact arithmetic.
- **Spaces matter.** " refund" and "refund" are different tokens. Trailing spaces in prompts can change outputs.

Industry use cases

Industry	Problem statement	How this concept solves it
Legal-tech	RAG chunks split by character count sometimes exceeded the embedding model's 512-token limit and got silently truncated.	Chunked documents by token count using the embedding model's own tokenizer, with a fixed token overlap.
E-commerce search	Users type misspellings ("iphnoe", "samsng") that a word-level system can't match.	Sub-word tokens still overlap with the correct word's pieces, so the model and embeddings stay robust to typos.
DevOps / Observability	Log summarisation hit limits because stack traces and UUIDs are token-heavy.	Pre-processed logs (masked UUIDs, deduplicated frames) before sending, reducing tokens ~60%.

3. Context

Definition: Context is all the information provided to the model in a single request that it can use when generating an answer: system instructions, conversation history, retrieved documents, tool results, and the user's question.



An LLM is a conditional probability machine

An LLM has two sources of knowledge: **parametric** (frozen in its weights at training time, possibly outdated, never includes your private data) and **contextual** (whatever you put in the prompt right now). The model is stateless: it does not remember the previous API call unless you send the history again. Everything "the bot knows about this customer" must be in the context.

$$P(y_1, y_2, \dots, y_m \mid \text{context}) = \prod_t P(y_t \mid \text{context}, y_1 \dots y_{t-1})$$

Every output token is conditioned on the whole context plus everything generated so far

Change the context and you change the probability distribution of every answer. This is why prompt engineering and RAG work without retraining the model.

Real example

Question: "Can I return this?" Without context the model gives a generic answer ("most stores allow 30 days"). With context containing the order (delivered 12 days ago, category: electronics) and the policy (electronics: 10-day return window), the model answers correctly: "This item is outside the 10-day window for electronics, but you can raise a warranty claim." Same model, same weights; only the context changed.

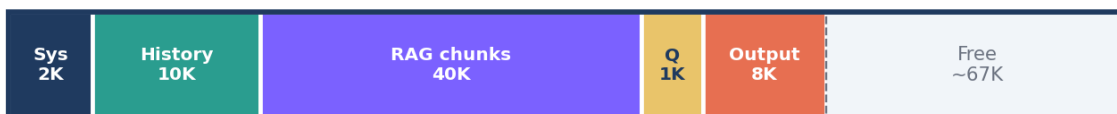
Industry use cases

Industry	Problem statement	How this concept solves it
Banking	Chatbot gave generic answers about loan status, frustrating customers.	Injected the customer's live loan record (via API/tool call) into context; answers became account-specific.
Software (coding assistants)	Code suggestions ignored the team's internal libraries and conventions.	IDE assistant places open files, imports and repo snippets into context, so suggestions use real function names.
Healthcare admin	Staff needed answers from a 400-page insurance policy manual.	Relevant manual sections are retrieved and placed into context (RAG), grounding answers in the actual policy.

4. Context Window

Definition: The context window is the maximum number of tokens a model can consider in one interaction. It is shared by the input (everything in the context) and the output the model generates.

Context window = 128K tokens (input + output must fit together)



Overflow $\Rightarrow$ oldest turns truncated or API error. Middle of very long prompts is recalled less reliably ('lost in the middle').

Why is there a limit at all?

Transformers use **self-attention**: every token compares itself with every other token to decide what is relevant. With n tokens that is $n \times n$ comparisons. Memory and compute therefore grow roughly quadratically, and the model was trained on sequences up to a certain length, so it cannot reliably generalise beyond it.

Hard constraint: $T_{\text{input}} + T_{\text{output}} \leq N_{\text{max}}$

Attention cost $\propto n^2 \Rightarrow (128\text{K} / 8\text{K})^2 = 16^2 = 256\times$ more pairwise work

Chunks needed = $\lceil (L - \text{overlap}) / (\text{chunk_size} - \text{overlap}) \rceil$
L = document length in tokens

Worked example: a 200-page contract $\approx 100,000$ words $\approx 133,000$ tokens. It does not fit in a 128K window with room for the answer. Options: (1) use a larger-window model, (2) chunk it: with 1,000-token chunks and 100-token overlap you get $\lceil (133,000 - 100) / 900 \rceil = 148$ chunks, then retrieve only the relevant ones (see Top-N), or (3) summarise section-by-section and combine summaries.

Practical realities

- **Bigger is not always better:** more tokens = more cost and latency, and models recall facts at the start and end of long prompts better than in the middle ("lost in the middle").
- **Chat memory is an illusion:** long conversations are managed by truncating old turns, sliding windows, or summarising earlier turns into a short memory.
- **Reserve output budget:** if input uses the whole window, the answer is cut off or the request fails.

Industry use cases

Industry	Problem statement	How this concept solves it
Legal / Procurement	Reviewing 150-page master service agreements manually took 2–3 days per contract.	Long-context model reads the full MSA in one pass and flags deviations from the standard template for lawyer review.
Customer support	After ~40 messages the bot "forgot" the order ID the user gave at the start.	Rolling summary of older turns + pinning key entities (order ID, customer tier) at the top of the context.
Enterprise meetings	2-hour call transcripts (~25K tokens) exceeded the older model's 8K window.	Moved to a 128K model for single-pass summarisation; for multi-day workshops, map-reduce summarisation across chunks.

5. Embedding

Definition: An embedding is a fixed-length vector of real numbers that represents the meaning of a piece of text (word, sentence, document, or image). Texts with similar meaning get vectors that point in similar directions.

From one-hot to meaning

Simplest way to turn words into numbers is **one-hot encoding**: with a 50,000-word vocabulary, "refund" = [0,0,...,1,...,0]. The problem: every pair of words is equally different. The dot product of "refund" and "reimburse" is 0, same as "refund" and "banana". The representation carries no meaning.

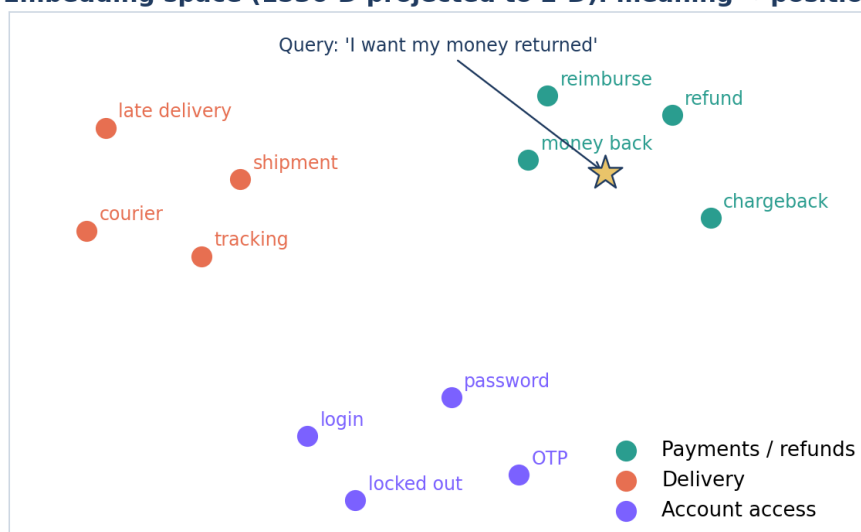
The fix comes from the **distributional hypothesis**: words that appear in similar contexts have similar meaning. A neural network is trained so that texts used in similar ways end up close together in a dense space of, say, 768 or 1536 dimensions. Each dimension is not human-readable, but together they encode meaning.

$$E : \text{text} \rightarrow \mathbb{R}^d \quad \text{e.g. } E(\text{"refund my money"}) = [0.021, -0.113, 0.487, \dots] \quad (d = 1536)$$

$$\text{vec}(\text{"king"}) - \text{vec}(\text{"man"}) + \text{vec}(\text{"woman"}) \approx \text{vec}(\text{"queen"})$$

Meaning becomes geometry: relationships become directions

Embedding space (1536-D projected to 2-D): meaning $\Rightarrow$ position



Notice that the query "I want my money returned" shares **zero keywords** with "refund" or "chargeback", yet lands right next to them. That is the entire value of embeddings: **search by meaning, not by keywords**.

Real example

An HR portal stores 3,000 policy paragraphs. Each paragraph is embedded once and stored in a vector database. An employee asks "Can I work from my hometown for a month?" The best matching paragraph is titled "Remote Work and Temporary Relocation", which contains none of the words "hometown" or "month". Keyword search (BM25) would likely miss it; embedding search finds it.

Industry use cases

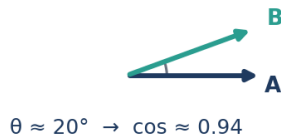
Industry	Problem statement	How this concept solves it
IT service desk	30% of tickets are duplicates of known issues, but worded differently, so agents re-solve them.	Embed every new ticket, compare with open incidents, auto-link duplicates to the parent incident.
Recruitment / HR-tech	Keyword matching rejected good candidates ("ML Engineer" vs "Data Scientist, deep learning").	Embed resumes and job descriptions, rank candidates by semantic match rather than exact keywords.
E-commerce	"Shoes for a beach wedding" returned no results because no product had that phrase.	Semantic product search over embedded titles and descriptions returns linen loafers and sandals.

6. Cosine Similarity

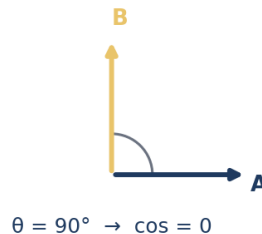
Definition: Cosine similarity measures how similar two vectors are by the cosine of the angle between them. Range: -1 (opposite) through 0 (unrelated) to +1 (same direction).

Cosine similarity measures the ANGLE between vectors, not their length

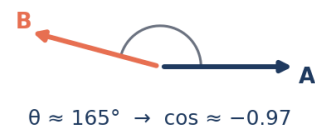
Similar meaning



Unrelated



Opposite direction



Why angle and not distance?

An embedding's **direction** encodes meaning; its **length** often reflects incidental things like text length or word frequency. A one-line review and a ten-line review saying the same thing should be considered similar. The angle ignores length and compares only direction, which is exactly what we want.

$$\cos(\theta) = (A \cdot B) / (\|A\| \times \|B\|) = \sum a_i b_i / (\sqrt{\sum a_i^2} \times \sqrt{\sum b_i^2})$$

If vectors are normalised ($\|A\| = \|B\| = 1$): $\cos(\theta) = A \cdot B$

and $\|A - B\|^2 = 2 - 2 \cdot \cos(\theta)$ (so Euclidean and cosine give the same ranking)

Most vector DBs normalise vectors and use the fast dot product

Worked example (toy 3-D embeddings)

A = "refund request" = [0.8, 0.6, 0.1], B = "money back please" = [0.7, 0.7, 0.2], C = "reset my password" = [-0.1, 0.2, 0.9]

- $A \cdot B = 0.56 + 0.42 + 0.02 = 1.00$; $\|A\| = \sqrt{1.01} = 1.005$; $\|B\| = \sqrt{1.02} = 1.010 \rightarrow \cos(A,B) = 1.00 / (1.005 \times 1.010) = 0.985$ (near-identical intent)
- $A \cdot C = -0.08 + 0.12 + 0.09 = 0.13$; $\|C\| = \sqrt{0.86} = 0.927 \rightarrow \cos(A,C) = 0.13 / (1.005 \times 0.927) = 0.14$ (unrelated intent)

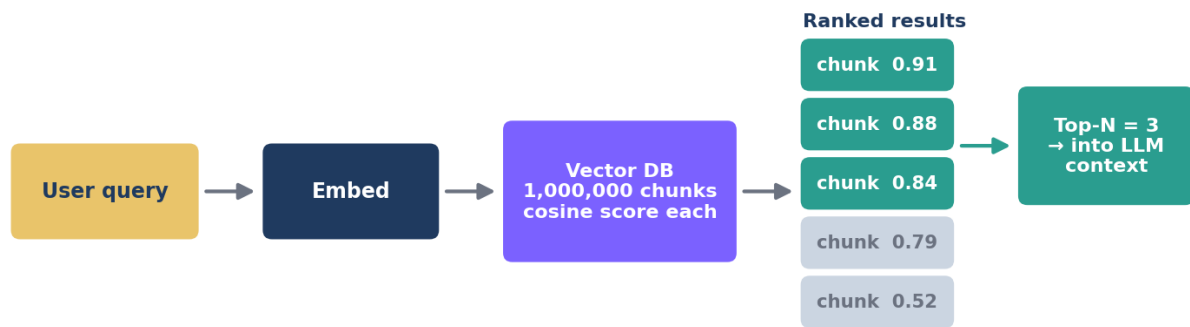
Engineering tip: Real embedding models rarely produce negative scores for text; unrelated sentences often score 0.1–0.4, related ones 0.7–0.9. Absolute thresholds are model-specific, so calibrate a threshold on labelled pairs from your own data instead of copying a number from a blog.

Industry use cases

Industry	Problem statement	How this concept solves it
Insurance	Fraud rings submit the same accident story with small wording changes across claims.	Embed claim narratives; pairs with cosine > 0.92 from different claimants are flagged for investigation.
Customer support	FAQ bot answered wrong questions when the match was weak.	Answer from FAQ only if top cosine $\geq$ calibrated threshold (e.g. 0.82); otherwise hand over to a human agent.
Media / Publishing	News aggregator showed 10 near-identical articles on the same event.	Cluster articles by cosine similarity and show one representative per cluster (de-duplication).

7. Top-N (Retrieval Ranking)

Definition: Top-N is the number of highest-scoring results kept after ranking candidates by similarity. In RAG, it is how many retrieved chunks are passed into the LLM's context. (Many vector databases call this parameter **top_k**.)



Why not send everything?

A knowledge base may have a million chunks, but the context window fits maybe a few dozen, and every extra chunk costs tokens and adds noise that can distract the model. So we score every chunk against the query (cosine similarity), sort, and keep only the best N. Choosing N is a **precision vs recall** trade-off.

$\text{Top-N}(q) = \text{the } N \text{ documents } d_i \text{ with the largest } \cos(E(q), E(d_i))$
 $\text{Precision@N} = (\text{relevant in top } N) / N$ $\text{Recall@N} = (\text{relevant in top } N) / (\text{total relevant})$
Small N → high precision, risk of missing facts. Large N → high recall, more noise and cost

Worked example: For a query there are 4 relevant chunks in the KB. With $N = 5$, the retriever returns 3 of them. $\text{Precision@5} = 3/5 = \mathbf{0.60}$, $\text{Recall@5} = 3/4 = \mathbf{0.75}$. With $N = 10$ it returns all 4: $\text{Recall@10} = \mathbf{1.0}$, but $\text{Precision@10} = 0.40$, so the LLM now reads 6 irrelevant chunks.

Common production pattern: retrieve wide, rerank narrow

Retrieve Top-50 using fast cosine search, then use a slower, more accurate **reranker** model to rescore those 50 and keep the final Top-5 for the LLM. You get high recall from stage 1 and high precision from stage 2.

Industry use cases

Industry	Problem statement	How this concept solves it
Enterprise HR bot	Answers were incomplete because the leave policy was split across 3 chunks but $N = 1$.	Raised N to 5 and added a reranker; answer completeness improved without flooding the context.
E-commerce	"Similar products" carousel needed fast, relevant suggestions.	Top-N = 12 nearest product embeddings by cosine, filtered by in-stock and price band.
Legal research	Lawyers need the few most relevant precedents out of 2M judgments.	Top-50 semantic retrieval → rerank → Top-10 shown with citations for human review.

8. Temperature

Definition: Temperature is a sampling setting that controls how predictable or varied a model's output is by sharpening or flattening the probability distribution over the next token.

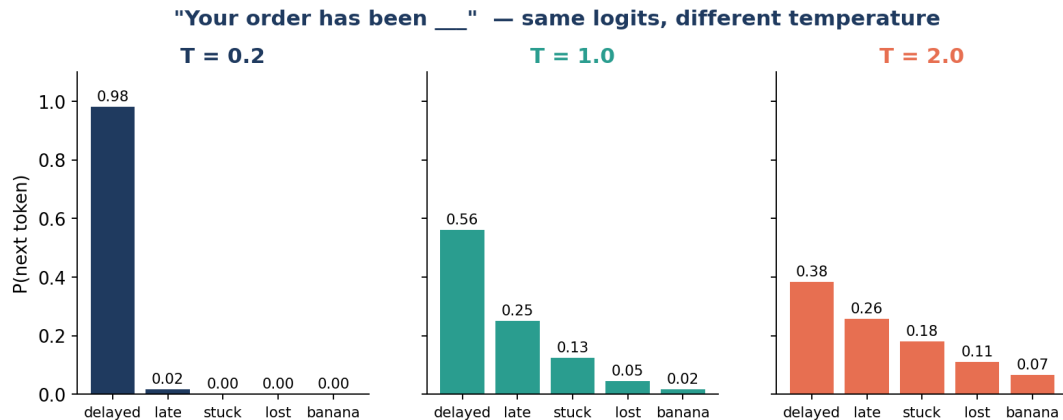
Where do probabilities come from?

At each step the model outputs a raw score (**logit**) z_i for every token in the vocabulary. The **softmax** function turns those scores into probabilities. Temperature T simply divides the logits before softmax.

$$p_i = \exp(z_i / T) / \sum_j \exp(z_j / T)$$

$T \rightarrow 0$: always pick the top token (greedy, deterministic)

$T = 1$: model's native distribution $T > 1$: flatter, more random



Worked example for "Your order has been ____" with logits delayed = 4.0, late = 3.2, stuck = 2.5, lost = 1.5, banana = 0.5:

- $T = 0.2$:** divide by 0.2 $\rightarrow$ 20, 16, 12.5, 7.5, 2.5. The gap is magnified: $P(\text{delayed}) = 0.98$. Nearly deterministic.
- $T = 1.0$:** $P = 0.56, 0.25, 0.13, 0.05, 0.02$. Natural variety.
- $T = 2.0$:** divide by 2 $\rightarrow$ 2.0, 1.6, 1.25, 0.75, 0.25. $P = 0.38, 0.26, 0.18, 0.11, 0.07$ for "banana". Creative, but nonsense becomes plausible.

Choosing temperature by task

Low temperature (0 – 0.3)	Higher temperature (0.7 – 1.0)
Data extraction, classification SQL / code generation Factual Q&A over documents (RAG) Anything that is parsed by code	Marketing copy, taglines Brainstorming, product naming Varied chatbot small-talk Generating diverse test data

Engineering tip: Temperature 0 reduces randomness but does not guarantee byte-identical outputs across runs (floating-point and batching effects). If you need reproducibility, also validate outputs with a schema and log prompts/versions.

Industry use cases

Industry	Problem statement	How this concept solves it
Accounts payable	Invoice field extraction gave different JSON keys on re-runs, breaking the ERP import.	Set temperature to 0 plus a strict JSON schema; output became consistent and machine-parseable.
Marketing / D2C brands	AI-generated ad copies all sounded the same, hurting A/B testing.	Temperature 0.9 to generate 20 varied variants, then humans shortlist the best 3.

9. Top-K (Sampling)

Definition: Top-K sampling restricts the model's next-token choice to the K most probable tokens, removes all others, renormalises, and then samples.

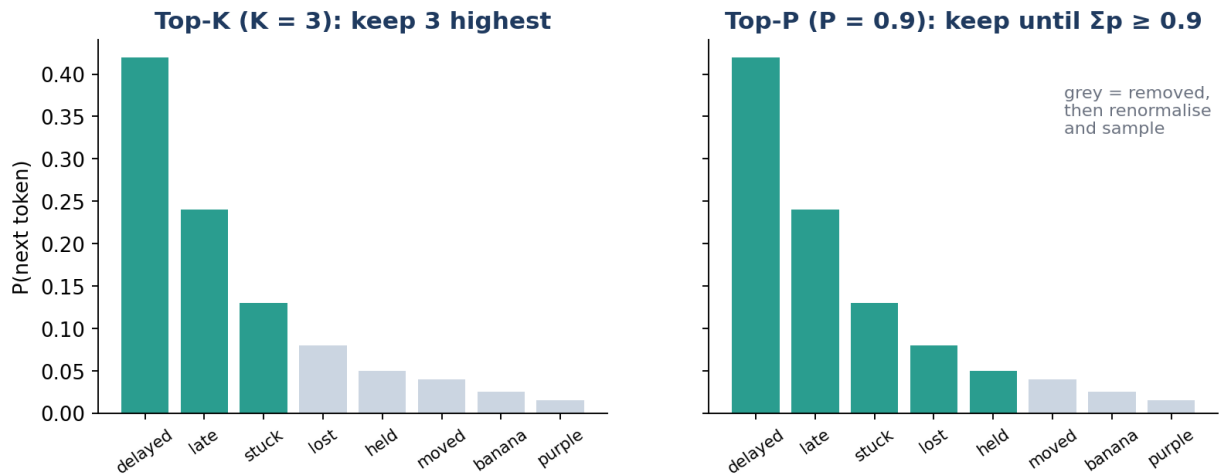
The long-tail problem

A vocabulary has ~100,000 tokens. Even if each absurd token has a tiny probability, together the long tail can hold a meaningful share of probability mass. Sampling 500 tokens from the full distribution means that, sooner or later, one weird token slips in and derails the sentence. Top-K cuts the tail off with a hard limit.

V_k = the K tokens with the highest p_i

$$p'_i = p_i / \sum_{j \in V_k} p_j \quad \text{if } i \in V_k, \quad \text{otherwise } p'_i = 0$$

Pipeline order: logits $\rightarrow$ $\div$ temperature $\rightarrow$ softmax $\rightarrow$ Top-K / Top-P filter $\rightarrow$ sample



Worked example (K = 3): kept probabilities 0.42, 0.24, 0.13 sum to 0.79. Renormalised: $0.42/0.79 = 0.53$, $0.24/0.79 = 0.30$, $0.13/0.79 = 0.17$. "banana" and "purple" can never be picked now.

Top-K vs Top-P (nucleus) — and vs Top-N

- **Top-K** keeps a fixed count. Weakness: when the model is very sure, $K = 50$ still allows 49 poor options; when it is unsure, $K = 3$ may cut good options.
- **Top-P** keeps the smallest set whose cumulative probability $\geq P$ (right chart: $0.42 + 0.24 + 0.13 + 0.08 + 0.05 = 0.92 \geq 0.9 \rightarrow 5$ tokens kept). The set size adapts to the model's confidence.
- **Top-N / top_k in retrieval** is a different stage: it ranks documents by similarity. Top-K sampling ranks next tokens by probability. Same idea ("keep the best few"), different place in the pipeline.

Industry use cases

Industry	Problem statement	How this concept solves it
Email / messaging apps	Smart-reply suggestions occasionally contained odd words, embarrassing users.	Small K (e.g. 5–10) with low temperature keeps replies safe and conventional.
Retail content	Generating 50,000 product descriptions; rare garbage tokens broke 1 in 200 pages.	Top-K = 40 and Top-P = 0.9 removed the long tail while keeping natural variety.
Gaming / Entertainment	NPC dialogue felt repetitive with greedy decoding.	Moderate temperature with Top-P sampling gave varied but coherent lines.

10. Sentiment Value (−1 to +1)

Definition: A sentiment value is a single number that summarises the emotional polarity of a text: -1 is strongly negative, 0 is neutral, and +1 is strongly positive.



Why a bounded, signed number?

Businesses need to **sort, threshold, average and trend** opinions across millions of texts. A label ("negative") can't be averaged or ranked by severity. A signed continuous value can: the sign gives direction, the magnitude gives intensity, and bounding to $[-1, 1]$ makes scores comparable across sources and time.

Three standard ways to compute it

- (1) From a classifier: $s = P(\text{positive}) - P(\text{negative})$
 - (2) Lexicon (VADER-style): $s = x / \sqrt{(x^2 + \alpha)}$, $x = \text{sum of word scores}$, $\alpha = 15$
 - (3) From star ratings: $s = (r - 3) / 2$, $r \in \{1 \dots 5\}$
- All three map naturally to the range $[-1, +1]$*

Worked example 1 (classifier): "Delivery was late and the box was damaged." Model outputs $P(\text{pos}) = 0.10$, $P(\text{neu}) = 0.20$, $P(\text{neg}) = 0.70 \rightarrow s = 0.10 - 0.70 = -0.60$.

Worked example 2 (lexicon): word scores "late" = -1.2, "damaged" = -1.9 $\rightarrow x = -3.1 \rightarrow s = -3.1 / \sqrt{(9.61 + 15)} = -3.1 / 4.96 = -0.62$.

Worked example 3 (aggregation): four reviews scored +0.9, -0.8, +0.4, -0.2 with helpful-votes weights 1, 3, 1, 1. Weighted mean = $(0.9 - 2.4 + 0.4 - 0.2) / 6 = -0.22$. The simple mean is +0.075, which hides the fact that the most-trusted review is strongly negative.

Pitfalls: Sarcasm ("Great, another 2-hour wait!" is not +0.8), negation ("not bad" is mildly positive), and mixed reviews ("Camera is superb, battery is terrible"). LLM-based or aspect-based sentiment, which scores each aspect separately, handles these far better than word lists.

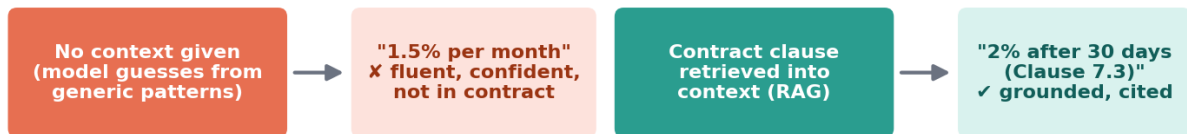
Industry use cases

Industry	Problem statement	How this concept solves it
Airlines / Travel	Angry social posts went unnoticed for hours and became PR incidents.	Score every mention in real time; $s < -0.6$ from high-follower accounts triggers priority escalation.
Contact centres	Supervisors could review only 2% of calls for quality.	Transcribe 100% of calls, track sentiment per turn; calls ending below -0.5 are routed for review and coaching.
Fintech / Trading	Analysts could not read thousands of news items per stock daily.	Daily average news sentiment per ticker used as a signal feature alongside price data.
E-commerce	Product teams didn't know which feature caused poor ratings.	Aspect-based sentiment (battery -0.7, camera +0.8) pinpoints the problem feature.

11. Hallucination

Definition: A hallucination is output that is incorrect or unsupported by the provided context or real-world facts, yet presented fluently and confidently as if it were true.

Q: What is the late-payment penalty in our 2024 vendor contract?



LLM optimises for the most probable next token, not for truth. Plausible $\neq$ correct.

Mitigations: grounding (RAG) · 'say I don't know' instruction · low temperature · citations · verification step.

Why do LLMs hallucinate?

The model was trained to predict the **most probable next token**, not to check facts. Probability measures how "typical" text looks, not whether it is true. When the answer is not in the weights or the context, the model does not naturally stop; it fills the gap with the most plausible-looking continuation, in the same confident tone as a correct answer.

Model optimises: $\hat{y} = \text{argmax } P(y \mid \text{context}) \leftarrow \text{fluency / plausibility}$

Business needs: y must be supported by evidence $\leftarrow \text{truth / groundedness}$

Hallucination rate = unsupported claims / total claims (measured on an eval set)

Main causes

- **Missing knowledge:** private, recent, or niche facts not in training data and not in context.
- **Pressure to answer:** the prompt implies an answer must exist ("Give the clause number for...").
- **High temperature / long outputs:** more sampling steps = more chances to drift.
- **Token-level weaknesses:** exact numbers, dates, IDs, and citations are pattern-completed, not recalled.

Real, well-known incidents

Mata v. Avianca (US, 2023): lawyers filed a brief containing court cases generated by ChatGPT that did not exist; the court sanctioned them. **Moffatt v. Air Canada (Canada, 2024):** the airline's chatbot described a bereavement-fare refund rule that did not match actual policy; the tribunal held the airline responsible for what its chatbot said. Lesson: **the company owns the output of its AI.**

Industry use cases (problem $\rightarrow$ mitigation)

Industry	Problem statement	How this concept solves it
Legal	Research assistant invented case citations.	Answers only from retrieved judgments (RAG), every claim must cite a source ID, and citations are verified against the database automatically.
Airline / Retail support	Bot promised refunds and discounts that policy doesn't allow.	Policy text in context, instruction "If the policy does not cover it, say you don't know and escalate", temperature 0.
Healthcare documentation	Clinical note summaries added medications never mentioned.	Second LLM pass checks each sentence against the transcript (groundedness check); unsupported sentences are removed and highlighted for the doctor.

12. Reasoning

Definition: Reasoning is the model's ability to work through multi-step problems (breaking them down, applying rules, calculating, and checking) rather than jumping straight to an answer.

Direct answer (one jump):



Step-by-step reasoning:



Why does "thinking step by step" help?

A model spends roughly a **fixed amount of computation per generated token**. A hard problem answered in one token gets one token's worth of compute. When the model writes intermediate steps, each step becomes part of the context for the next step, so later tokens are conditioned on already-computed partial results. Writing the steps is literally giving the model more computation and a scratchpad. Modern "reasoning" or "thinking" models are trained to do this automatically before answering.

Direct: $P(\text{answer} \mid \text{question})$

With reasoning: $P(\text{answer} \mid \text{question}) = \sum_r P(r \mid \text{question}) \cdot P(\text{answer} \mid \text{question}, r)$

Chain reliability: $P(\text{all } n \text{ steps correct}) = p^n$ e.g. $0.95^5 = 0.77$, $0.95^{10} = 0.60$

r = a reasoning path. Self-consistency: sample several paths and take the majority answer

The last line is critical for engineers: even at 95% accuracy per step, a 10-step chain is fully correct only 60% of the time. That is why production systems add **verification steps**, **tool use** (calculator, code execution, database lookups) and **majority voting** for critical decisions.

Worked example: insurance claim

"Policy covers 80% after a ₹5,000 deductible, max payout ₹1,00,000. Claim is ₹32,000. What is the payout?"

- Step 1 (extract): claim = 32,000, deductible = 5,000, coverage = 80%, cap = 1,00,000
- Step 2 (apply rule): eligible = 32,000 – 5,000 = 27,000
- Step 3 (calculate): payout = 0.80 × 27,000 = **₹21,600**
- Step 4 (check): 21,600 < 1,00,000 cap → final answer ₹21,600

A direct one-jump answer commonly errs by applying 80% before the deductible ($0.8 \times 32,000 - 5,000 = 20,600$). Explicit steps expose and prevent that error, and they make the result auditable.

Industry use cases

Industry	Problem statement	How this concept solves it
Lending (NBFC / Banks)	Loan eligibility depends on income, FOIR, credit score and existing EMIs; errors caused bad approvals.	Step-by-step reasoning with calculations done by a tool, and each rule check logged for auditors.
Finance operations	Reconciling invoices, POs and payments with partial shipments and credit notes.	Model reasons line-by-line, matching items and computing differences, then flags mismatches.
Software engineering	Debugging failures across multiple services and logs.	Reasoning model forms hypotheses, checks logs/code for each, eliminates causes, proposes a fix.

13. Zero-shot Prompting

Definition: Zero-shot means asking the model to perform a task using only instructions, without giving any examples of inputs and expected outputs.

Why does it work at all?

During pre-training on huge amounts of text, and later instruction tuning, the model has already seen countless examples of summarising, translating, classifying, and answering. A clear instruction activates that existing capability. No training, no examples: just describe the task.

$$\hat{y} = \operatorname{argmax} P(y \mid \text{instruction}, x)$$

Output depends only on the instruction and the input x

Zero-shot prompt	Model output
Classify the sentiment of this review as Positive, Negative or Neutral. Review: "Battery died in 2 days."	Negative

Where it works well and where it breaks

- **Works well:** generic, well-known tasks such as summarisation, translation, general sentiment, language detection, rewriting, and common-sense Q&A.
- **Breaks down:** company-specific label sets ("P1-Billing-Refund-Partial"), strict output formats, domain jargon, and borderline cases where your business rule differs from common sense.
- **Why teams start here:** it is the fastest to build, uses the fewest tokens, and gives a baseline to measure improvements against.

Industry use cases

Industry	Problem statement	How this concept solves it
Global support	Tickets arrive in 20+ languages and must be routed to the right regional team.	Zero-shot language detection and translation; no examples needed because the model already knows these tasks.
Corporate communications	Executives need 3-line summaries of long internal reports.	Zero-shot summarisation with a clear instruction on length and audience.
Compliance	Quickly check whether outgoing emails contain personal data (phone, PAN, Aadhaar).	Zero-shot detection as a first-pass filter, with regex and a human review for flagged cases.

14. Few-shot Prompting

Definition: Few-shot means including a small number of worked examples (input → desired output) in the prompt to demonstrate the task, the labels, and the exact output format.

In-context learning

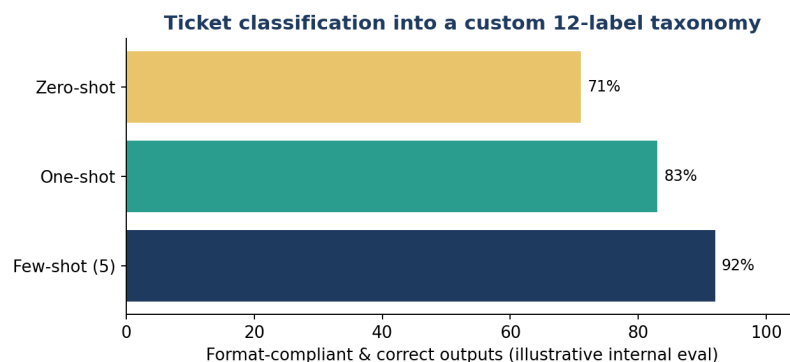
The model continues patterns. When the context contains "input → output" pairs, the most probable continuation for a new input follows the same pattern. The model's weights do **not** change; it is learning from the context for this one request only. Examples communicate what words alone struggle to: edge cases, tone, labels, and format.

$$\hat{y} = \operatorname{argmax} P(y \mid \text{instruction}, (x_1, y_1), (x_2, y_2), \dots, (x_k, y_k), x)$$

Extra prompt cost $\approx k \times (\text{tokens per example}) \times (\text{number of requests})$
k = number of examples (typically 2–8)

Few-shot prompt (custom labels)	Model output
Label the ticket. "Charged twice for order 88" → BILLING_DUPLICATE "Refund only half credited" → BILLING_PARTIAL_REFUND "App crashes on UPI page" → TECH_PAYMENT_UI Ticket: "Got ₹200 back, paid ₹500" →	BILLING_PARTIAL_REFUND

Zero-shot would likely answer "Refund issue", a reasonable label that your routing system doesn't recognise. The examples teach the exact taxonomy and format.



Cost example: 5 examples × 60 tokens = 300 extra tokens per call. At 1,00,000 calls/day that is 30M extra input tokens/day, so choose examples carefully and consider caching fixed prompt prefixes where your provider supports it.

Best practices

- Cover **every label** and the tricky edge cases; balance examples so the model isn't biased towards one class.
- Keep examples **consistent in format**; the model copies format very faithfully, including mistakes.
- For large label sets, **retrieve the most similar examples** per request using embeddings + cosine similarity (dynamic few-shot).

Industry use cases

Industry	Problem statement	How this concept solves it
Accounts payable (India)	GST invoices from 500 vendors have different layouts; fields must map to a fixed JSON.	3 examples of invoice text → exact JSON (GSTIN, HSN, taxable value, CGST/SGST) gave consistent structured output.
Customer support	Routing into a 40-label internal taxonomy failed with zero-shot.	Dynamic few-shot: fetch the 5 most similar past labelled tickets as examples for each new ticket.

15. Putting It Together: One Request, Every Concept

A customer types: "Paid ₹500 but only got ₹200 back, this is ridiculous!" Here is what happens inside a production support assistant:

- **Tokenization / Token:** message becomes ~20 tokens; tokens are counted for cost and window budget.
- **Sentiment value:** $s = -0.72 \rightarrow$ below the -0.6 threshold, so the ticket is tagged high-priority.
- **Embedding + Cosine similarity:** the message is embedded and compared with the refund-policy knowledge base.
- **Top-N:** best 4 policy chunks (cosine 0.88, 0.85, 0.81, 0.79) are selected.
- **Context + Context window:** system rules + order record + 4 chunks + few-shot examples + question are assembled and checked to fit the window with output reserved.
- **Few-shot:** examples make the model output a fixed JSON with label BILLING_PARTIAL_REFUND.
- **Reasoning:** the model computes the pending amount ($500 - 200 = ₹300$) and checks the policy's refund timeline.
- **Temperature 0 + Top-K / Top-P:** predictable, parseable output with no long-tail tokens.
- **Hallucination control:** answer must cite the policy chunk; if the policy doesn't cover the case, the bot escalates to a human.